

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
по дисциплине
SDR

по теме:

Программная реализация детектора временной ошибки(синхронизация приемника и передатчика)
на SDR. Написание функций петли (контура) синхронизации

Студент:
Группа ИА-331

К.А Любимов

Предподаватели:
Лектор
Практик
Практик

Калачиков А.А
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

1 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ДЕТЕКТОРА ВРЕМЕННОЙ ОШИБКИ(СИНХРОНИЗАЦИЯ ПРИЕМНИКА И ПЕРЕДАТЧИКА) НА SDR. НАПИСАНИЕ ФУНКЦИЙ ПЕТЛИ (КОНТУРА) СИНХРОНИЗАЦИИ

I [Ссылка на GitHub](#)

Цель практики:

Написать функцию которая будет автоматически синхронизировать примник и передатчик.

Краткие теоретические сведения

Оптимальный приемник выполняет прием дискретных сигналов с минимальной вероятностью ошибки. Строится оптимальный приемник по схеме на основе корреляторов или согласованных фильтров.

Оптимальный приемник выбирает наиболее близкое принятому сигналу $r(t)$ из набора возможных колебаний $s_i(t)$. Это может выполняться вычислением расстояния

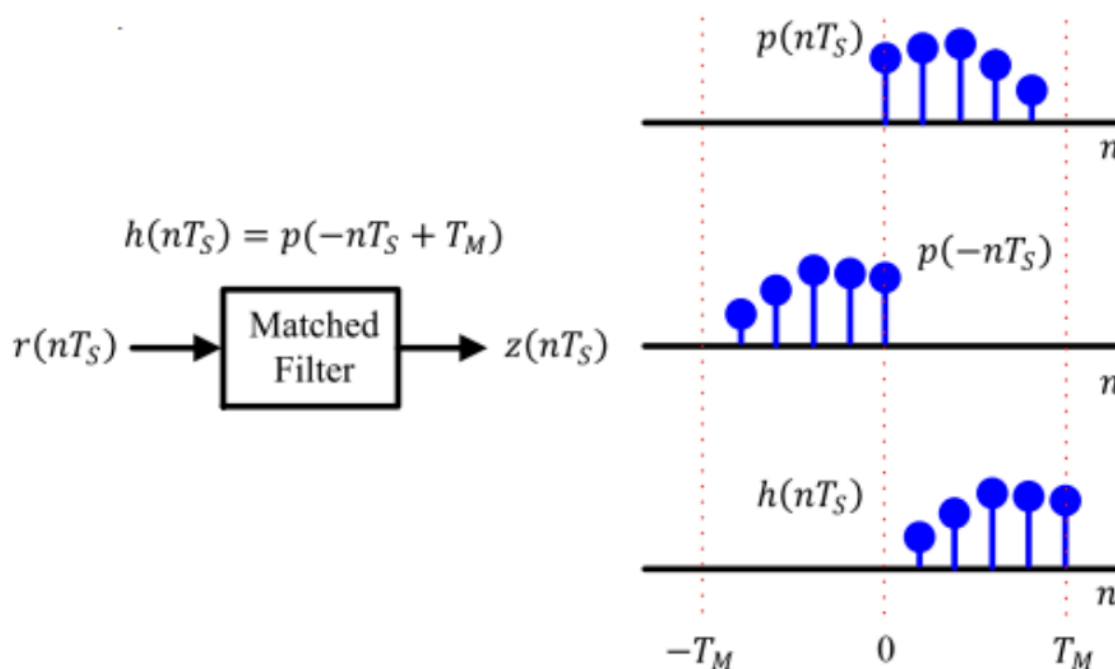


Рис. 1 — Matched filter

Если использовать формирующий фильтр с прямоугольной формой ИХ, то форма ИХ согласованного фильтра также является прямоугольной. Сигнал на выходе СФ вычисляется как свертка отсчетов входного сигнала и ИХ согласованного фильтра.

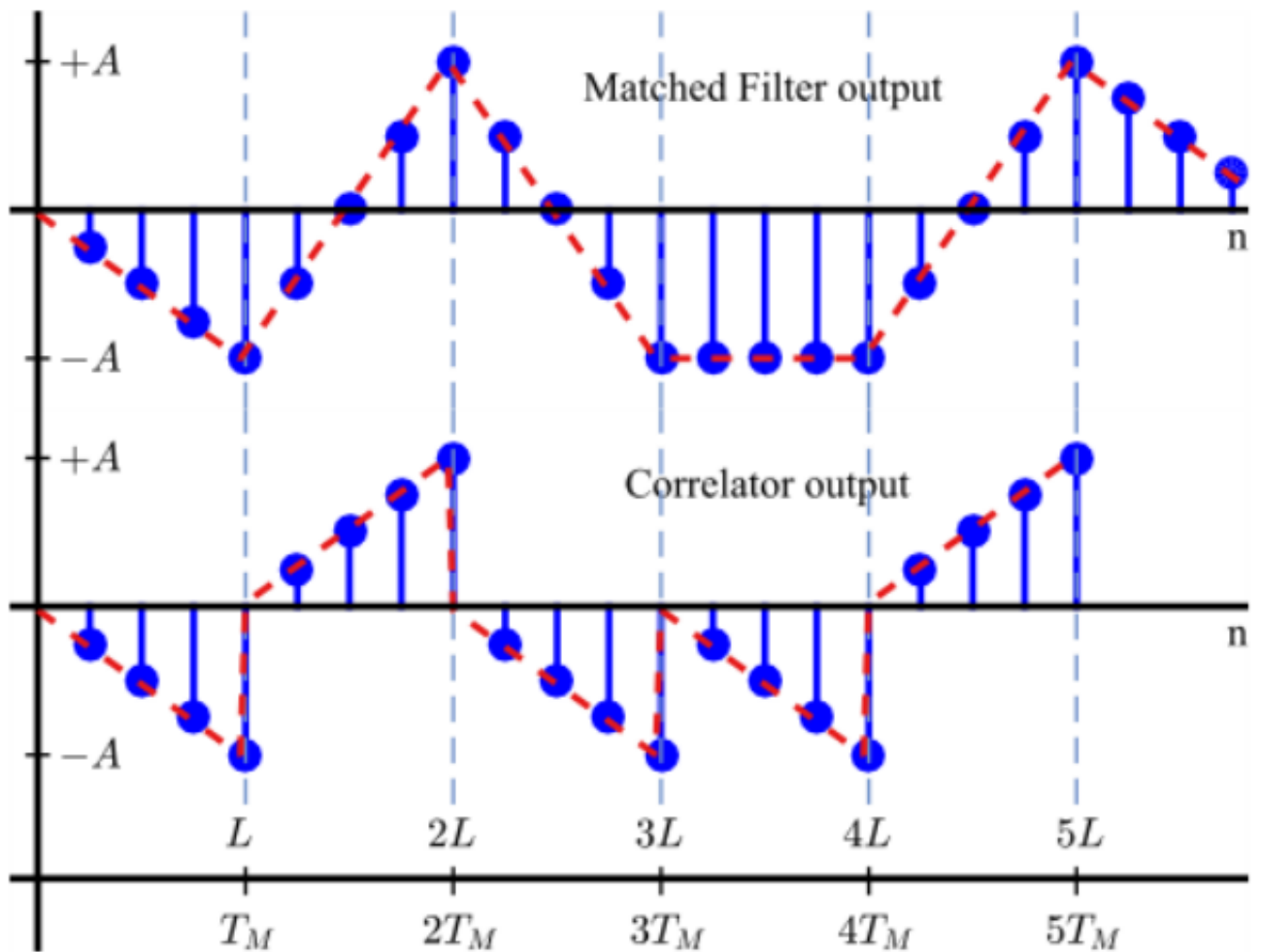


Рис. 2 — Matched filter/correlator output

Демодулятор должен сделать отсчет сигнала на выходе СФ точно в момент максимума сигнала на выходе СФ. Если время взятия отсчета не совпадает с максимумом, то невозможно принять надежное решение о передаваемом символе.

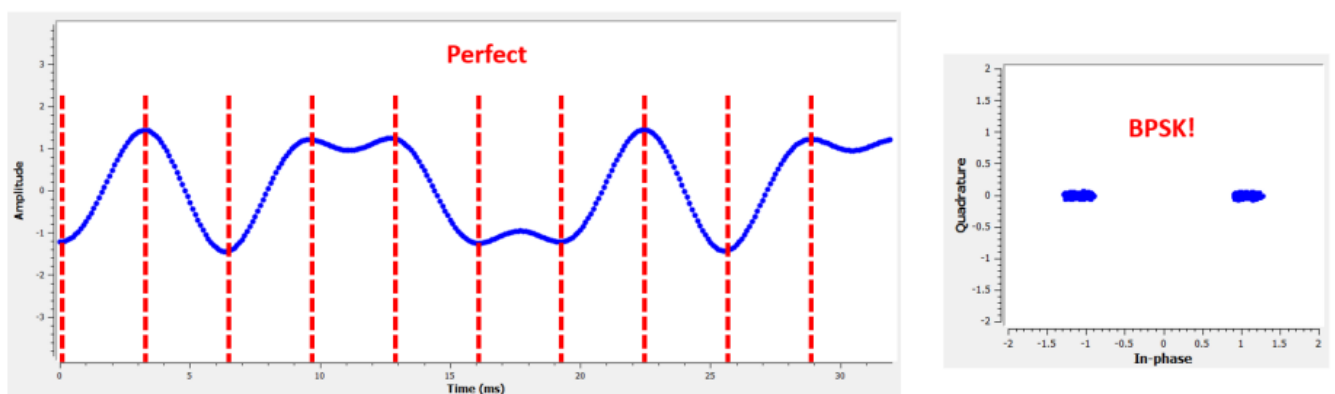


Рис. 3 — Идеальный отсчет

Время взятия отсчета (symbol timing) совпадает с максимумом сигнала на выходе СФ. При смещении времени взятия отсчета берутся значения с выхода СФ не в оптимальные моменты времени, что приводит к нарушению возможности точного определения амплитуды символа.

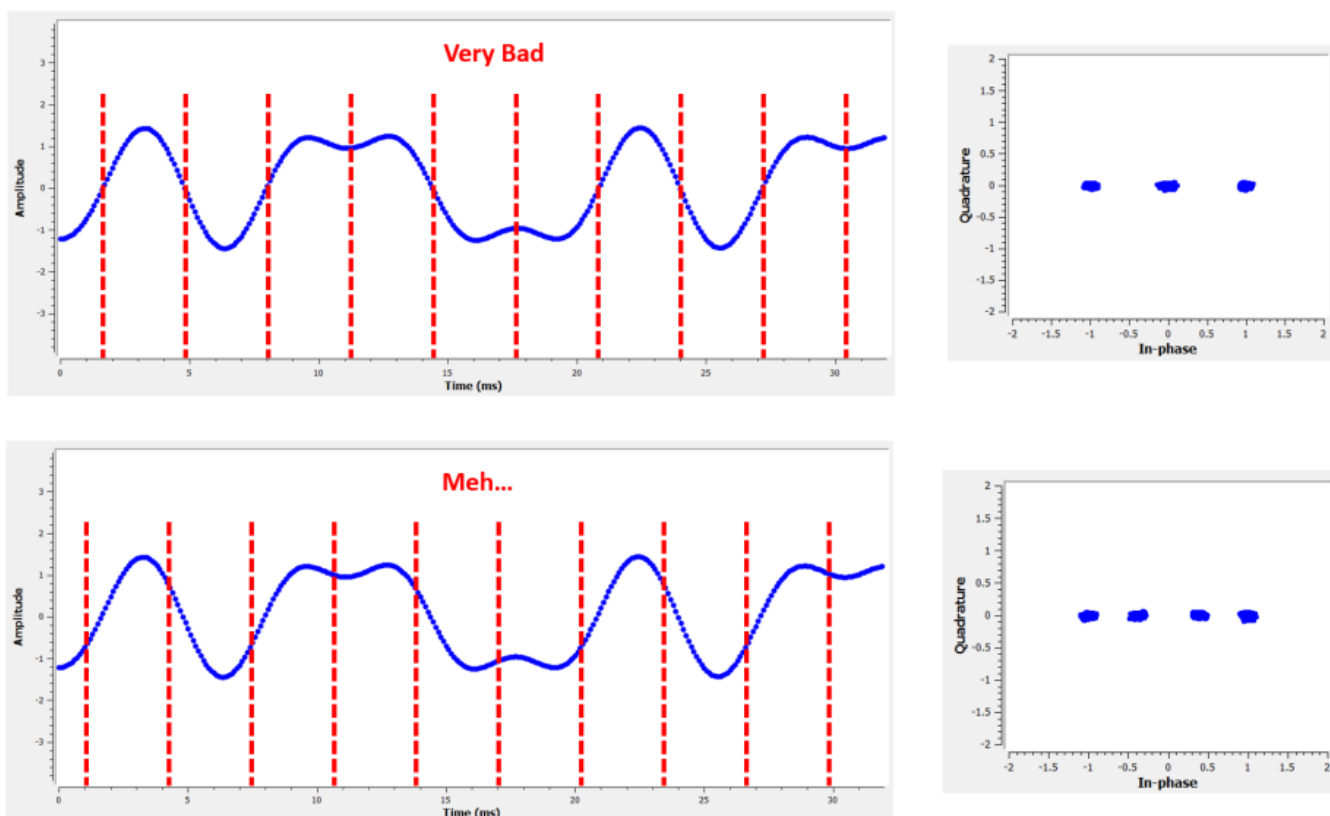


Рис. 4 — Плохой отсчет

Детектор временной ошибки - TED

Для поиска нужного отсчета сначала вычисляется ошибка положения символа при помощи детектора временной ошибки. Детектор использует набор отсчетов и вычисляет ошибку ТЕД $e(n)$, величина которой зависит от смещения максимума выхода СФ. В таблице приведены выражения для вычисления величины ошибки по отсчетам сигнала с выходы СФ.

TED Method	Expression
Zero-crossing (decision-directed)	$e(k) = x((k - 1/2)T_s + \hat{\tau})[\hat{a}_0(k - 1) - \hat{a}_0(k)] + y((k - 1/2)T_s + \hat{\tau})[\hat{a}_1(k - 1) - \hat{a}_1(k)]$
Gardner (non-data-aided)	$e(k) = x((k - 1/2)T_s + \hat{\tau})[x((k - 1)T_s + \hat{\tau}) - x(kT_s + \hat{\tau})] + y((k - 1/2)T_s + \hat{\tau})[y((k - 1)T_s + \hat{\tau}) - y(kT_s + \hat{\tau})]$
Early-late (non-data-aided)	$e(k) = x(kT_s + \hat{\tau})[x((k + 1/2)T_s + \hat{\tau}) - x((k - 1/2)T_s + \hat{\tau})] + y(kT_s + \hat{\tau})[y((k + 1/2)T_s + \hat{\tau}) - y((k - 1/2)T_s + \hat{\tau})]$
Mueller-Muller (decision-directed)	$e(k) = \hat{a}_0(k - 1)x(kT_s + \hat{\tau}) - \hat{a}_0(k)x((k - 1)T_s + \hat{\tau}) + \hat{a}_1(k - 1)y(kT_s + \hat{\tau}) - \hat{a}_1(k)y((k - 1)T_s + \hat{\tau})$

Рис. 5 — TED

Зависимость сигнала на выходе TED от величины смещения символа (временной ошибки) называется S-curve, для детектора Гарднера она имеет форму

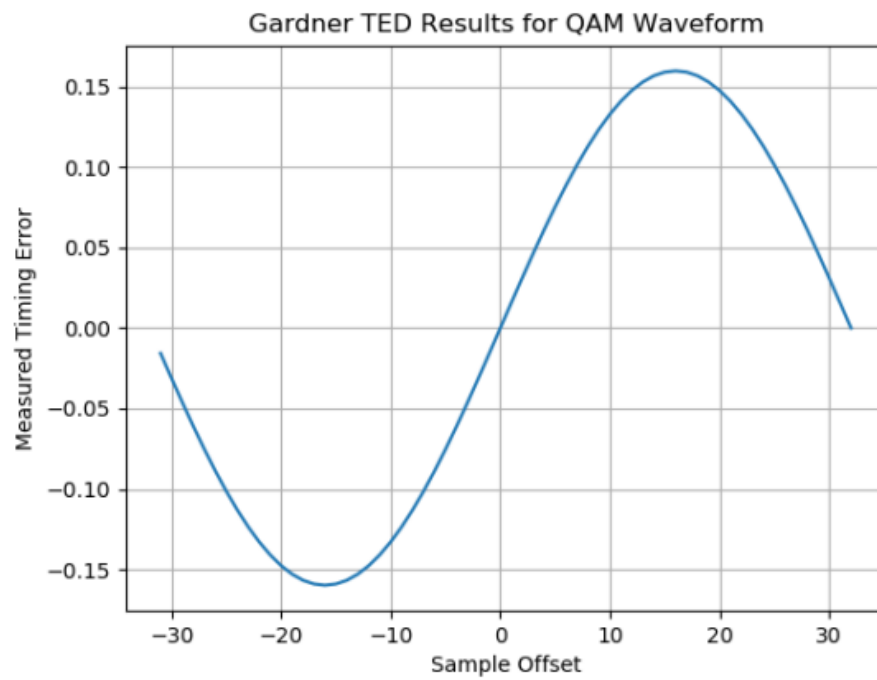


Рис. 6 — Форма TED

Фильтр контура

Фильтр выполнен в виде пропорционально-интегрирующего звена по схеме

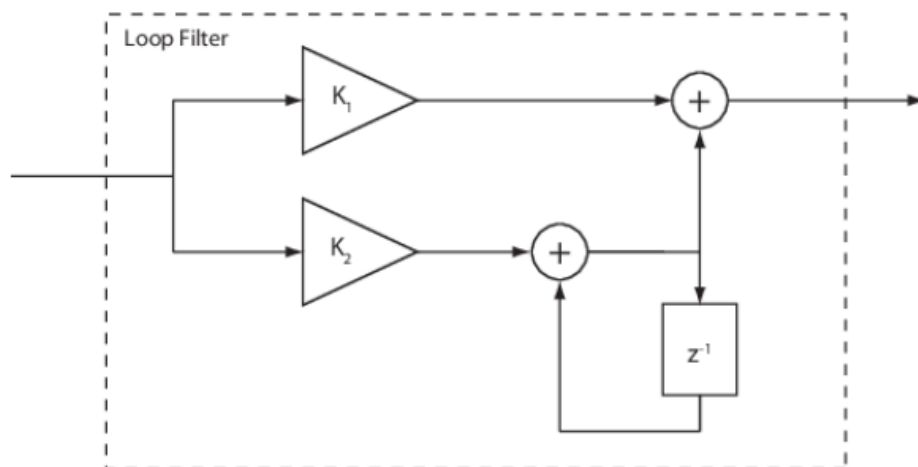


Рис. 7 — Форма TED

Параметры фильтра

$$K_1 = \frac{-4\zeta\theta}{(1 + 2\zeta\theta + \theta^2)K_p}$$

and

$$K_2 = \frac{-4\theta^2}{(1 + 2\zeta\theta + \theta^2)K_p}.$$

The interim term, θ , is given by

$$\theta = \frac{\frac{B_n T_s}{N_{sps}}}{\zeta + \frac{1}{4\zeta}},$$

where:

- N_{sps} is the number of samples per symbol.
- ζ is the damping factor.
- $B_n T_s$ is the loop bandwidth (B_n) normalized to the symbol rate (T_s).
- K_p is the detector gain.

Рис. 8 — Параметры фильтра

Программно фильтр реализуется в общем цикле по отсчетам сигнала на выходе СФ.

Начальные значения переменных в ветвях K1 и K2 - p1 и p2 равны 0. Сами величины K1 и K2 вычисляются перед циклом. Нормированная полоса фильтра контура $B_n T_s = 0.01$. Также перед циклом задается начальное смещение $offset = 0$ (номер отсчета символа, один из 10).

В цикле по отсчетам сигнала на выходе СФ вычисляется текущее значение ошибки e для текущего смещения $n = offset$;

$$e = (I(n+Nsp+Ns) - I(n+Ns)) * I(n+Nsp/2+Ns) + (Q(n+Nsp+Ns) - Q(n+Ns)) * Q(n+Nsp/2+Ns).$$

Если e - текущая величина ошибки на выходе детектора ошибки (на данном индексе отсчета сигнала на выходе СФ), то в ветви K1 переменная $p1 = e * K1$. В ветви K2 переменная $p2 = p2 + p1 + e * K2$. Эта переменная p2 имеет смысл задержки, дробного смещения ($offset$) на длительности символа. Тут же, если $p2 > 1$, ее сбрасывают

$$\text{while}(p2 > 1) p2 = p2 - 1; \text{end.}$$

После этого определяется уточненное смещение в отсчетах ($offset$), для этого нужно умножить $p2$ на $Nsps$ и округлить произведение до ближайшего целого, $offset = \text{round}(p2 * Nsps)$, это будет смещение на данной итерации (оптимальный отсчет текущего символа, индекс в массиве отсчетов сигнала на выходе СФ). На следующем символе снова вычисляется величина ошибки и после фильтрации определяется смещение, по которому находится следующий индекс отсчета $offset + Nsps$. Таким образом контур сводит ошибку к минимуму и определяются оптимальные индексы в массиве отсчетов символов сигнала на выходе СФ.

Выполнение

Напишем функцию Гарднера, которая будет автоматически искать $offset$

Листинг 1.1 — Функция Гарднера

```

1 import numpy as np
2 def gardner(complex_symbols_saw):
3     p1 = 0
4     p2 = 0
5     offset = 0
6     BnTs = 0.01
7     Nsp = 10 # Samples per symbol
8     zeta = np.sqrt(2)/2
9     teta = (BnTs/Nsp)/(zeta + 1/(4*zeta))
10    css = complex_symbols_saw
11    Kp = 0.002
12    K1 = (-4 * zeta * teta)/((1 + 2 * zeta * teta + teta ** 2) * Kp)
13    K2 = (-4 * teta ** 2)/((1 + 2 * zeta * teta + teta ** 2) * Kp)
14    list_of_offset = []
15
16    for n in range(0, len(css)-22):
17        err = (np.real(css[n + Nsp + Nsp]) - np.real(css[n + Nsp])) * np.real(css[n +
18            Nsp + Nsp//2]) + (np.imag(css[n + Nsp + Nsp]) - np.imag(css[n + Nsp])) *
19            np.imag(css[n + Nsp + Nsp//2])
20        err = np.real(err)
21        p1 = err * K1
22        p2 = p2 + p1 + err * K2
23        while p2 > 1:
24            p2 = p2 - 1
25        while p2 < -1:
26            p2 = p2 + 1
27        offset = np.round(p2 * Nsp)
28        list_of_offset.append(offset)
29
30    return int(offset), list_of_offset

```

Используем эту функций в нашем коде

Листинг 1.2 — Функция Гарднера

```

1 from matplotlib import pyplot as plt
2 import numpy as np
3 from auto_offset import gardner
4
5 rx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
6     int16)
7
8 samples = []
9
10 for x in range(0, len(rx), 2):
11     samples.append((rx[x] + 1j * rx[x+1])/np.max(rx))
12
13 plt.figure()
14 plt.scatter(np.real(samples), np.imag(samples))
15 plt.axhline()
16 plt.axvline()
17
18 impulse = np.ones(10)
19
20 I = np.real(samples)
21 Q = np.imag(samples)
22
23 i_convolve = np.convolve(I, impulse)
24 q_convolve = np.convolve(Q, impulse)

```

```

24
25 i_con_dec = []
26 q_con_dec = []
27 list_offset = []
28
29 offset, list_offset = gardner(i_convolve + 1j * q_convolve)
30 print(offset)
31
32 plt.figure()
33 plt.title("offset")
34 plt.plot(np.arange(len(list_offset)), list_offset)
35 plt.ylim([-20, 20])
36 plt.xlim([4060, 4560])
37
38 for i in range(offset, len(i_convolve), 10):
39     i_con_dec.append(i_convolve[i])
40     q_con_dec.append(q_convolve[i])
41
42 # Убираем шумы
43 # i_con_dec = [x for x in i_con_dec if not (abs(x) <= 1)]
44 # q_con_dec = [x for x in q_con_dec if not (abs(x) <= 1)]
45
46 print("length i_con_dec: ", len(i_con_dec))
47 print("length q_con_dec: ", len(q_con_dec))
48
49 plt.figure()
50 plt.subplot(2,1,1)
51 plt.scatter(i_con_dec, q_con_dec)
52 plt.axhline()
53 plt.axvline()
54 plt.subplot(2,1,2)
55 plt.xlim([5000, 5500])
56 plt.ylim([-30, 30])
57 plt.scatter(np.arange(len(i_convolve)), i_convolve)
58 plt.scatter(np.arange(len(q_convolve)), q_convolve)
59
60 plt.show()

```

Вывод

Написал, программную реализация детектора ошибки, исполнил в коде схему Гарднера.